Lab 1 – From Using Agents to Building Them

Agentic Software Engineering · Winter Semester 2026/27 · Paderborn University

In this lab you first use a professional coding agent (OpenCode), then build a small agent yourselves, then look under the hood at two things every agent depends on: retrieval (RAG) and tool calling.

At a glance

- Groups of 5 students
- About 10 hours per student (about 50 hours per group)
- Python 3.11+
- All LLM calls go through the UPB AI Gateway (`ai-gateway.upb.de`). It only works in the university network or via the university VPN.
- All material is released at once on PANDA as a ZIP file.

Dates

Mon, Oct 26	Release of the complete lab on PANDA (all four tasks)
Tue, Oct 27	Lecture: Tool use and function calling, Part I
Tue, Nov 3	Lecture: Tool use and function calling, Part II
Sat, Nov 14, 23:59	**Deadline** for everything: material ZIP and video, both via PANDA

Points

Part	Points	Suggested time (group)
Task 1 – Meet your agent (OpenCode)	6	Setup + Task 1: ~7 h
Task 2 – Build your own coding agent	14	~12 h
Task 3 – RAG under the microscope	14	~11 h
Task 4 – Tools for agents	16	~13 h
Video presentation (max. 10 minutes)	**50**	~5 h
Total	**100**	

Suggested order: Tasks 1–3 first. Task 4 builds on the lecture of Oct 27; its containment part also uses the lecture of Nov 3 (see the notes at the task).

Rules on AI usage

- You may use AI assistants and coding agents for Tasks 2–4, including to write code.
- Task 1 is different: you write **no** code yourselves. You only provide content and talk to the agent.
- You are responsible for everything you submit. Every group member must be able to explain every part of the submission; "the AI wrote that part" is not an explanation.
- Include a short AI usage statement in your video: which tools you used, and for what.
- Never commit or submit API keys.

Setup

Every group member needs:

- Python 3.11+ (we recommend `uv`) and git
- OpenCode (`opencode.ai`)
- A personal API key for the UPB AI Gateway. Get it before you start:
  1. Service Portal `sp.upb.de` → Benutzerverwaltung → Weitere Dienste → Lehre und Studium → KI-Chat-Oberfläche → Beantragen (approved automatically)
  2. Connect to the university network or the VPN (`hilfe.uni-paderborn.de/VPN_einrichten`)
  3. `ai-gateway.upb.de` → log in → "API-Key erstellen". Keys expire; create a new one when calls fail with 401.

Material: download `lab1.zip` from PANDA and unpack it. It contains everything for all four tasks. Setup of the Python environment and the gateway key: `README.md` in the root folder. Setup of OpenCode: `task1_opencode/README.md`. Gateway details: `hilfe.uni-paderborn.de/AI-Gateway`.

Models: use `openai.gpt-6-luna` for all tasks (chat, agents, tool calling) and `kiconnect.qwen3-embedding-8b` for embeddings. If your budget is used up, switch to the free fallback `kiconnect.qwen3.8-27b`. Costs, limits and the response cache are described in `README.md`.

Budget: every student has $20 per month on the gateway. With the recommended model, the whole lab costs a group a few dollars. `llm_client.py` logs the cost of every call.

Task 1 – Meet your agent: a team website built with OpenCode

6 points

Goal: Get to know a real coding agent. Learn how to steer it, where it helps, and where it goes wrong.

Steps

1. **Setup** (each group member): follow `task1_opencode/README.md` up to the hello world. Save a screenshot of it as `task1_opencode/screenshots/hello_<firstname>.png`.
2. **Content** (each group member, no code): fill in `content/<firstname>.md` from the template and add a picture to `content/images/`. A photo, avatar or drawing is fine.
3. **Let the agent build the site** (group): one standalone file `task1_opencode/site/index.html`
   - HTML + CSS + JavaScript only, nothing loaded from the internet
   - introduces the team and every member, with everyone's picture embedded in the file
   - responsive (phone and desktop), day/night mode toggle
   - a section with your team's shared view on AI and the future of software engineering
   - funny and creative
4. **Save every conversation:** export all OpenCode sessions (see the README) to `task1_opencode/conversation/`, and note prompts, tokens and cost per session.

Rules

- You do not write or edit any HTML/CSS/JS yourselves. Every code change comes from the agent.
- Several sessions and several group members taking turns are fine.
- Check the site with `task1_opencode/check_site.py`. Tip: think about how the images get into the file, and whether the model should write that part itself.

> **Lecture note.** What actually happens when OpenCode "runs" a command or edits a file (the model only proposes, the agent program executes and asks for permission), and why every turn re-sends the whole conversation, is covered in the lecture *Tool use and function calling, Part I* on **Tue, Oct 27**. You can do Task 1 before that; the lecture helps you explain what you observed in your video.

Grading (6 points)

- 4 pts – Website meets the requirements
- 1 pt – Complete conversation logs, and it is clear that no code was written by hand
- 1 pt – Session statistics (prompts, tokens, cost) and what you observed about them (in the video)

Task 2 – Build your own (small) coding agent

14 points

Goal: Build a Python program that creates the same kind of website on its own, using retrieval and a real agent loop. No frameworks (no LangChain, LlamaIndex, agent SDKs).

The twist: the style guide

- The website must also follow the fictional "UPB Agentic SE Web Style Guide" (`task2_own_agent/style_guide/`, 55 documents). No model knows it from training.
- Some documents are deprecated versions of rules.
- You can't paste the whole guide into the prompt. Your agent has to retrieve (limits in the README).

What to build (full specification: `task2_own_agent/README.md`)

1. `rag.py` – chunk and embed the style guide; `search(query, k)` returns the most similar chunks with their source documents
2. `agent.py` – an agent loop with a JSON action protocol that you design yourselves (no native tool calling, that is Task 4):
   - actions: `search_style_guide`, `read_profiles`, `write_site` (saves the site and runs the checker), `finish` (only accepted when the checks pass)
   - robust against invalid model output, with a step limit
   - human in the loop: after finishing, a user can give feedback and the agent continues
3. One command runs the agent end to end, with a complete log of the conversation in `task2_own_agent/logs/`

> **Lecture note.** Retrieval is covered in the RAG lecture. Your JSON action protocol is your own design; how providers turn tool calls into a guaranteed format (native function calling, constrained decoding) is the topic of the lecture on **Tue, Oct 27**. You don't need it to build Task 2, but it is useful for comparing your protocol with native tool calling in your video.

Grading (14 points)

- 4 pts – Agent loop works end to end: protocol, invalid output handled, step limit, finish only after passing checks, human feedback
- 4 pts – RAG: chunking, embeddings, retrieval with sources, limits respected
- 3 pts – Resulting website: Task 1 features + style guide compliance (we run `check_site.py`)
- 1 pt – Complete logs of at least one full run
- 2 pts – README: how to run it, design decisions, known limitations

Task 3 – RAG under the microscope

14 points

Goal: Compare retrieval approaches systematically on a larger corpus. Understand when each approach wins and when it fails.

Task 3 does not depend on Task 2. You may reuse your Task 2 code. All approaches are covered in the RAG lecture.

Dataset and scripts (details, file formats and hints: `task3_rag/README.md`)

- PaderQueue, a fictional message queue: 932 documents (configuration, error codes, CLI, release notes, incidents, forum threads, runbooks)
- 150 dev questions with ground truth in 5 categories: identifier, paraphrase, multi-hop, version, unanswerable
- 100 test questions without ground truth
- `evaluate.py` (context recall/precision) and `judge.py` (LLM judge: faithfulness, answer relevance, correctness)

What to do

1. Implement and evaluate these retrieval configurations:
   - R1 – BM25, on the same chunks as R3
   - R2 – fixed-size chunking + dense embeddings
   - R3 – recursive chunking (headings, paragraphs, sentences) + dense embeddings
   - R4 – hybrid search: normalized and weighted combination of R1 and R3. Justify your weight.
   - R5 – R4 plus ONE of: re-ranking with a cross-encoder, query rewriting or HyDE, semantic chunking
2. Chunk size: for R3, compare 3 settings (e.g. 3 chunk sizes with the same overlap).
3. Retrieval evaluation with `evaluate.py`, overall and per category.
4. End-to-end evaluation: for your two best configurations, generate answers for 50 dev questions (`judge.py --sample 50`) and score them with `judge.py`. Check 10 judge decisions by hand: do you agree?
5. Cost and latency per configuration: indexing time, latency per query, approximate tokens per query.
6. Failure analysis: 5 questions your best system gets wrong. Retrieval failure or generation failure, and why?
7. Recommendation: which configuration would you deploy, and why?
8. Submit predictions for the test questions with your best configuration (`task3_rag/predictions_test.jsonl`).

Grading (14 points)

- 6 pts – R1–R5 implemented correctly
- 3 pts – Evaluation done correctly: per-category table, chunk size comparison, cost/latency, judge check
- 3 pts – Failure analysis and recommendation. We grade the reasoning, not the leaderboard rank.
- 2 pts – Test set predictions in the right format and better than the R1 baseline

Task 4 – Tools for agents: function calling, validation, containment

16 points

Goal: Build a safe tool layer for a debugging agent. Use it to let an agent find and fix bugs in a small repository.

Task 4 does not depend on Tasks 1–3.

> **Lecture note.** This task builds on two lectures that take place after the release:
>
> - **Tue, Oct 27 – Tool use and function calling, Part I:** the basis for the whole task: native function calling and tool schemas, why you still validate when the format can be guaranteed (constrained decoding), shaping tool output for the model, authorisation and least privilege, idempotency, and tool output as untrusted input (prompt injection).
> - **Tue, Nov 3 – Part II:** stateful tools, tool granularity and selection, tool definitions as attack surface, and real path-prefix and symlink vulnerabilities in a filesystem tool server, which is directly relevant to your containment.
>
> We recommend starting Task 4 after Oct 27 and finishing the containment part after Nov 3.

The toy repository: tinyledger

- A small Python library + CLI for splitting group expenses, with several bugs: some make tests fail, one is described in `ISSUE.md`.
- Treat the repository like code from the internet. Not everything in it is trustworthy.

What to build (interface, tool list, limits and workflow: `task4_tools/README.md` and `tool_contract.py`)

1. `tools.py` – a `ToolRegistry` with the tools `list_files`, `read_file`, `search_code`, `run_tests`, `edit_file` (`run_cli` is optional)
   - bounded, structured outputs (e.g. `run_tests` returns a summary, not raw pytest output)
2. **Validation** – every call is checked against its JSON schema before it runs. Invalid JSON, unknown tools, wrong or missing arguments never crash; they return an error the model can act on.
3. **Containment** (least privilege) – paths stay in the workspace (incl. symlinks), writes only below `src/`, no shell, timeouts, no API keys in subprocesses, output limits, every tool call logged.
4. `agent.py` – a loop with native function calling. Task for the agent: "Make all tests pass and fix the problem described in ISSUE.md. Do not modify tests."
   - it must also run with the provided mock model (`--mock`), which sends broken and malicious tool calls
   - run it with a real model and submit the trace and the final diff
5. Your tool layer must pass the public tests (`task4_tools/tests/`). We also run hidden adversarial tests.

Grading (16 points)

- 3 pts – Tool interface and schemas: clear descriptions, sensible granularity, bounded structured outputs
- 3 pts – Validation and error handling: public tests + our hidden malformed-input tests
- 5 pts – Containment: our hidden adversarial tests (path escapes, hanging processes, huge outputs, protected files, environment leakage)
- 3 pts – Agent run: trace + diff. Bugs fixed without touching the tests. A partial fix with a good trace still earns points.
- 2 pts – Analysis in the video: what untrusted content did the agent see, how did it react, and which of your controls mattered

Video presentation

50 points

Instead of a live presentation, every group records **one video of at most 10 minutes**. It is half of your grade, so plan it like a deliverable, not an afterthought.

- **Length:** at most 10:00 minutes. Anything after 10:00 is not graded.
- **Format:** MP4, at least 720p, with audio. Screen recording with voice-over is fine; slides, live demos and screenshots are all fine.
- **Speakers:** every group member presents a part.
- **Evidence:** show your own results (tables, logs, traces, screenshots). Every number you show must be reproducible from your submission.

Required content, in this order

Part	What to show	Time (guide)
1. Intro	Group name, members, a screenshot of your Task 1 website	~0:30
2. Task 1 – Using an agent	Screenshots: desktop + mobile, day + night. Session statistics: prompts, tokens, cost. One moment where the agent went wrong and how you steered it back.	~1:30
3. Task 2 – Your own agent	One diagram: loop, actions, RAG, checker, human feedback. One short annotated step: what was sent to the model, what came back, what your code did.	~2:00
4. Task 3 – RAG	Table: configurations × metrics, broken down by question category. Failure analysis and your recommendation.	~2:00
5. Task 4 – Tools	The list of tools, with read-only vs. write marked. A table: threat → control → where in the code. A condensed trace of fixing one bug, and what happened with untrusted content.	~2:30
6. Wrap-up	Lessons learned, AI usage statement, who did what	~1:00

Grading (50 points)

- 8 pts – Task 1 part
- 12 pts – Task 2 part
- 12 pts – Task 3 part
- 12 pts – Task 4 part
- 6 pts – Wrap-up (lessons learned, AI usage statement, contributions), clarity, and keeping to the 10 minutes

For each task part we grade: is the required content there, are your explanations correct, and do you answer the guiding questions with evidence from your own runs rather than general statements.

Guiding questions

You can't answer all of these in 10 minutes. Use them to decide what to focus on: pick the questions where your own results have something to show.

Task 1

- How many prompts did you need? Which prompts worked best, and why?
- How did token usage change over a long session? Why?
- What did OpenCode do on its own (reading files, running commands)? Did it ask for permission?
- How did the images end up in the HTML file?

Task 2

- What is the difference between your program and a fixed pipeline? Where does the model make decisions?
- How did you choose chunk size, overlap and k? How did you check that retrieval returns the right rules?
- What happens when the model produces invalid JSON, repeats itself, or never calls finish?
- How does your context grow over the steps? What would happen with 100 steps?
- Which style guide rules did the agent get wrong, and why?

Task 3

- Which approach wins in which question category, and why?
- What did chunk size change?
- Where does your best system fail: in retrieval or in generation?
- How much do you trust the LLM judge?
- What do the better approaches cost in latency and tokens?

Task 4

- Why did you split the tools the way you did? Why no generic "run shell command" tool?
- The model's output already follows the schema. Why do you still validate?
- Which attacks does your containment stop? Which ones does it not stop?
- What untrusted content did the agent see? How did it behave?
- What would you need to change before letting this agent work on a real company repository?

Submission (due Sat, Nov 14, 23:59)

Submit **two separate uploads on PANDA**, one per group:

1. **The material:** one ZIP file `group-XX_lab1.zip` with all your code, results and logs, in the structure below.
2. **The video:** `group-XX_lab1_video.mp4`, uploaded separately. Do not put the video into the ZIP.

```
group-XX/
  README.md                   group members, matriculation numbers, how to run everything
  CONTRIBUTIONS.md            who did what (short bullet list per person)
  task1_opencode/
    content/                  profiles + images
    screenshots/              hello world screenshot of every member
    site/index.html
    conversation/             exported OpenCode sessions
  task2_own_agent/
    rag.py, agent.py, ...     your code
    README.md
    logs/run_XX.jsonl
    site/index.html
  task3_rag/
    code
    results/                  evaluation outputs (json/csv)
    predictions_test.jsonl
  task4_tools/
    tools.py, agent.py, ...
    logs/trace.jsonl
    final.diff                don't include workspace/
```

- Don't include: `.env`, API keys, virtual environments, embedding caches larger than 50 MB.